角雷达信息安全 · TI AWR2944 HSM 深度解析

60 分钟完整讲稿

> **讲者**：PeterZhu **时长**：60 分钟 **对象**：嵌入式固件工程师 / 系统工程师
> **配套 PPT**：`AWR2944_Security_Training.pptx` (47 张)

开场 · Slide 1 (约 30 秒)

各位同事下午好，我是 PeterZhu。今天这 60 分钟，我们聊一个之前在内部反复被问、但一直没讲透的话题——**角雷达 ECU 的信息安全，尤其是 TI AWR2944 里面那颗 HSM 到底是怎么工作的。**

这份材料在原版 32 页的基础上，修订了 10 处内容，新增 15 张专讲 TI HSM 和公司 HSM-RT 落地的幻灯片。讲完之后，大家应该能清晰回答三个问题：**X.509 证书里那一堆 OID 到底是什么**、**镜像验签 7 步到底在验什么**、以及**我们公司的 BM/PFBoot/CustBoot 在 TI 真实启动链里处于哪一段。**中间有疑问随时打断。

目录导航 · Slides 2–4 (约 45 秒)

Slide 2 · 整个培训分 5 章：第一章信息安全基础铺垫 10 分钟，第二章密码学核心技术 12 分钟，第三章把视角收敛到角雷达 ECU 10 分钟，第四章也是最核心的一章——AWR2944 HSM 实战 20 分钟，最后 8 分钟讲公司 HSM-RT 是怎么在 TI 之上落地的。

Slides 3 & 4 · 这是细化目录。**特别说明**：原版把第三章误标成了"二"，这是我修订的第一处；同时把原来塞在第四章的公司落地内容单独拆成了第五章，让 TI 通用机制和博世定制内容彻底分开——这条分层贯穿整个培训，待会儿讲到 Slide 40 你们会看到为什么这样分至关重要。

好，进入第一章。

第一章 · 信息安全基础 (10 分钟)

Slide 5 · 第一章封面 (15 秒)

从这里开始，我们聊一个很多功能安全工程师容易忽略的范式——**为什么汽车不能照搬企业 IT 的安全思路。**

Slide 6 · 信息安全的六大支柱 (90 秒)

信息安全的地基就这六个字母：**CIA + AAA。**

- **C 机密性**：只有授权方能读，典型实现 AES-128 做对称加密、TLS 做传输加密
- **I 完整性**：数据没被改过，靠 SHA-256 或 HMAC
- **A 可用性**：系统别宕机、别被 DoS，对 ECU 来说就是看门狗加冗余
- 下面三个 A——**认证、授权、可追溯**，对应 X.509 证书、RBAC 权限、以及 Secure Log

这六个支柱不是摆设，后面每一项技术选型都能映射到其中一个或多个。比如我们做 SecOC，本质是解决 I + A1 + A3；做 Secure Boot，本质是解决 I 加信任链。**先记住这个坐标系，后面不会迷路。**

Slide 7 · 汽车信息安全的五大特殊性 (100 秒)

为什么不能把 IT 那一套拿过来？五个原因：

1. **安全即人命**——被黑的是汽车，物理后果是人命。这决定了我们对误报容忍度接近零
2. **长生命周期**——整车 15 年以上，比你手机长 5 倍。意味着密码学算法必须可升级，今天你用 RSA-2048，2030 年必须能换到 RSA-3072 或 PQC
3. **资源受限**——MCU 只有 KB 级 RAM、MHz 级主频，整套 TLS 栈塞不进来，必须硬件加速
4. **实时性约束**——CAN 帧 ms 级响应，加密不能引入明显延迟，这就是 HSM 存在的根本原因
5. **分布式信任**——上百个 ECU + 云端 + 路侧，零信任架构是基本假设

这五条加起来，就是为什么汽车需要专门的信息安全体系。

Slide 8 · 三个改写汽车安全史的案例 (90 秒)

三个里程碑案例：

2015 年 Jeep Cherokee——Miller 和 Valasek 通过 Uconnect 娱乐模块远程入侵，跨 CAN 控制方向和刹车。FCA 紧急召回 140 万辆。这起事件**直接催生了 UNECE WP.29 立法**。注意，我特意**去掉了原版里"1 亿美元损失"这个没有权威来源的数字**——法规催生效果本身就足够有力，不需要靠虚数字。

2016 年腾讯科恩——通过浏览器漏洞链，远程开特斯拉天窗和刹车。14 天内 OTA 修复，但更重要的是，这件事推动了全行业代码签名 + Secure Boot 的落地。

2022 年 Honda Rolling-PWN——无钥匙系统滚动码重放，SDR 录制解锁+启动。推动行业从 Rolling Code 升级到挑战-响应的 Smart Key 2.0。

三个案例，三次行业升级。 每一次都是"事故驱动标准"。

Slide 9 · 汽车攻击面全景 (75 秒)

从攻击者视角看，入口分四个域：**云端、通信、车内、终端**，一共 13 类典型入口。

- 云端：OTA 服务器、TSP 后台、API 接口
- 通信：4G/5G 蜂窝、V2X、WiFi/蓝牙
- 车内：CAN-FD、以太网骨干、OBD-II——**雷达 ECU 就在这个域**
- 终端：TBox、智能钥匙、充电口

攻击者通常从云或通信层渗透，最终落到车内。**角雷达虽然不面向外部，但作为 ADAS 决策的数据源，一旦被篡改，后果直接等同于 ADAS 决策失效**——这就是我们今天讨论它的意义。

Slide 10 · 法规与标准全景 (90 秒)

我把法规分成两条轨道：**强制法规**和**推荐技术标准**。

强制法规——不合规就拿不到型式认证、就不能上市：

- **UN R155（CSMS）**——要求 OEM 建立全生命周期的网络安全管理流程
- **UN R156（SUMS）**——软件更新管理体系，每次 OTA 升级都要能追溯和回滚
- **中国 GB 44495**——2024 年发布，2026 年 1 月开始强制，对标 R155 但加国密要求

注意，我**把原版笼统的"WP.29"拆成了 R155 和 R156**——这两份法规在工程上落地的侧重点完全不同，混着讲很容易让人以为只有一份。

推荐技术标准——是落地法规的具体抓手：**ISO/SAE 21434** 定义 TARA 分析方法，几乎所有 OEM 的安全流程都是这份的变体；**ISO 24089** 是 R156 的技术指南；**ISO 21177 + 26262** 用于 V2X 和功能安全的交叉考量。

记住一个映射关系：**法规告诉你"必须做"，标准告诉你"怎么做"。**

第一章小结 · 过渡到第二章 (10 秒)

基础铺到这里。接下来 12 分钟进入技术内核——具体**用什么数学工具、什么证书体系**来解决这些需求。

第二章 · 信息安全核心技术 (12 分钟)

Slide 11 · 第二章封面 (15 秒)

这一章基本上就是密码学在车规场景的应用版，节奏会稍快，但每一个概念后面都有工程落点。

Slide 12 · 对称 vs 非对称 (110 秒)

两条路线：**对称密码速度快、密钥分发难；非对称相反。**

- 对称：AES-128/256 是主力，HSM 里硬件加速，1MB 数据毫秒级加解密完成。**典型应用**：镜像加密、TLS 数据面
- 非对称：RSA-3072 / ECC-P256 / Ed25519 / 国密 SM2。速度比对称慢 100 到 1000 倍，但**不需要预共享密钥。典型应用**：证书验签、密钥协商

一个重要更新——**我把原版里的 RSA-2048 升级到了 RSA-3072。**NIST 已经明确 RSA-2048 在 2030 年后弃用，后量子密码 ML-KEM / ML-DSA 即将进入车规标准，大家在新项目上直接从 RSA-3072 起步，给 PQC 迁移留出时间窗。

雷达里的用法：**MEK 做对称，加密 OTA 包；SMPK 做非对称，验签 Boot 链。**

Slide 13 · 完整性三剑客 (100 秒)

防篡改三种工具，越往右安全性越强，但成本也越高：

- **Hash**（哈希）——防误传不防伪造，任何人都能算出 hash。镜像下载校验就用这个
- **HMAC**——在 Hash 基础上加对称密钥，**防伪造但需要双方共享密钥**，SecOC 帧认证用这个
- **数字签名**——非对称版本，**防伪造 + 不可抵赖**，X.509 证书就是靠它签出来的

一条经验：**HMAC 性能好适合每帧；数字签名慢一些，只用在事件级事件上**，比如启动验签、OTA 包验签。如果你看到有同事在每一帧 CAN 上加数字签名，那是架构错了。

Slide 14 · PKI 与 X.509 证书 (120 秒)

这张是第四章的理论铺垫，一定要听清。

信任链——Root CA 自签名、离线保管，万年信任之锚；Intermediate CA 在线签发设备证书，可撤销；Device Cert 是最终烧到 eFuse 里的那个。验签的时候反向溯源：拿到 Device Cert，用 Intermediate 公钥验它、再用 Root 公钥验 Intermediate，三层都通过才算可信。

X.509 v3 结构——9 个核心字段：Version、Serial、Issuer、Validity、Subject、Public Key、**Extensions**、Signature。

特别看 Extensions 这一栏——这是 X.509 标准留给厂商的"自定义区"，TI 就是在这里塞了 7 个 OID 来保存 boot 元数据。**今天第四章的核心机密就藏在这个字段里**，Slide 30 到 33 会详细拆。

Slide 15 · Secure Boot 启动链 (100 秒)

Secure Boot = 验证过再执行。四级递进：

ROT (Root of Trust) → SBL → OS → App。每一级都被前一级验签，只要有一级失败，整链停机。

三条黄金法则：

1. **Verify Before Execute**——跳转前先验签，别"先跑起来再说"
2. **Anchor to Hardware**——信任锚必须在 ROM 或 eFuse，软件能改就等于没加
3. **Anti-Rollback**——旧版本镜像不能重新刷回来，用 SWRV 单调计数器

顺便说一下，**原版这里残留了"演讲提示"模板占位符**，我已经删掉——这是第三处修订。

Slide 16 · 安全测试方法论 (100 秒)

六种手段，我按测试阶段从早到晚排：

- **SAST**——源码级静态分析，Coverity / Polyspace，代码 review 阶段跑
- **DAST**——运行时黑盒测试，Burp Suite，集成测试阶段
- **Fuzzing**——随机或智能变异输入，AFL++ / Peach，**持续集成 + 长跑**
- **PenTest**——白帽视角，端到端验证防御链，型式认证前
- **HW Side-Channel**——侧信道攻击，ChipWhisperer，HSM 模块单独测
- **Fault Injection**——故障注入，VCglitcher，Tier-1 实验室才玩得起

实战经验：**Fuzzing 是性价比最高的一种**——跑上一周，能抓出 80% 的协议解析漏洞。

第二章小结 · 过渡到第三章 (15 秒)

好，数学工具和证书体系都有了，下一步把视角收回到我们的工作对象——**角雷达 ECU 在整车架构里的位置、威胁、以及该满足的标准。**

第三章 · 角雷达信息安全 (10 分钟)

Slide 17 · 第三章封面 (15 秒)

这一章是"标准到工程"的衔接。讲完这 6 张，你应该能回答：**"我们这颗雷达，为什么需要 HSM？"**

Slide 18 · 角雷达的角色与威胁 (100 秒)

先说它的位置——**FL/FR/RL/RR 四个角，全部通过 CAN-FD 或 100M 车载以太网连到 ADAS 域控。**

有个很常见的误解：**"角雷达就是边缘节点、数据简单、安全要求低"。**错。它是 ADAS 决策的"眼睛"，目标列表被改一个字段，域控可能就不会刹车。

右侧是标准的 **STRIDE 威胁分析模型**：

- **S**poofing——伪造雷达 ECU，上报虚假障碍

- **T**ampering——刷入恶意固件，屏蔽真实障碍触发碰撞
- **R**epudiation——事故后否认特定数据来源
- **I**nformation Disclosure——导出标定数据 / 算法 IP 泄漏
- **D**enial of Service——高频干扰 → 总线饱和、雷达失效
- **E**levation of Privilege——经诊断口提权刷写 ProductionApp

这六个威胁不是学究分类，是**给你做 TARA 时的 checklist。**

Slide 19 · 雷达 ECU 必须满足的标准矩阵 (90 秒)

7 条硬性要求，按层级排：

- **整车 OEM 层**：R155 CSMS + R156 SUMS
- **流程层**：ISO/SAE 21434 全开发流程合规，输出 Cybersecurity Case
- **软件层**：ISO 24089 升级包结构
- **国密要求**：GB 44495 + GB/T 32918 (SM2)——中国市场绕不开
- **芯片层**：EVITA Full 或 SHE+，带硬件 HSM
- **调试层**：ISO 14229 UDS 0x29 安全访问服务

每一条背后都意味着一堆文档和验证工作。这也是为什么 ECU 项目的 Cybersecurity 工程师越来越多——**活儿不是变少了，是变全栈了。**

Slide 20 · 从威胁到对策的需求映射 (90 秒)

上一张是纵向的标准栈，这张是横向的映射：**每一个 STRIDE 威胁，对应哪种 CIA 属性、用什么工程对策、落到哪个硬件能力。**

念一下关键对：

- Spoofing → Authentication → 证书 + TIFEK → **X.509 + HSM 硬件密钥**
- Tampering → Integrity → Secure Boot + 镜像签名 → **BMPK 验签链**
- DoS → Availability → SecOC 帧认证 + 看门狗 → **HSM 加速 CMAC**

这张表是 TARA 的输出，也是接下来第四章硬件讨论的输入。**右侧每一项硬件能力，我们都会在第四章逐一展开。**

Slide 21 · 雷达 ECU 整体安全方案 (90 秒)

三层防御：从上到下是**运行时安全 → 启动安全 → 硬件根。**

- **运行时**：SecOC 帧认证、Secure Storage 存密钥和标定、Secure Log、TLS 通道
- **启动时**：HSM ROM 验签、R5 SBL 验签、App 验签、反回滚 SWRV
- **硬件根**：TI HSM Cortex-M4F、eFuse OTP 存密钥、TRNG 真随机数、DTHE 加速

读图口诀：自上而下是攻击者面对的防御纵深；自下而上是信任传递的路径。**硬件根要是被攻破，上面两层再完美也没用**——这就是为什么 HSM 是整个方案的基石。

Slide 22 · AWR2944 / Gen2 Radar 平台概览 (100 秒)

TI 第二代车规雷达 SoC 关键参数：

- **45nm RFCMOS 工艺**

- **主控**：双核 Cortex-R5F 可配 Lockstep 功能安全双核
- **DSP**：C66x @ 360MHz，做雷达信号处理
- **HSM**：Cortex-M4F + DTHE，**我们今天的主角**
- **ASIL**：B/D Capable，满足 ISO 26262 功能安全
- **LC 生命周期**：GP / HS-FS / HS-SE 三态——第 26 张单独讲

修订点：原版叫"AWR 系列"范围太宽，AWR1xxx 一代根本没有 HSM。我改成了"AWR2944 / Gen2 Radar"，这样讨论的对象就明确了。

第三章小结 · 过渡到第四章 (15 秒)

外围铺垫结束。下面 20 分钟是今天的重头戏——**我们要钻到那颗 Cortex-M4F 的 ROM 里，看 TI 到底是怎么设计这套机制的。**

第四章 · AWR2944 HSM 实战 (20 分钟)

Slide 23 · 第四章封面 (15 秒)

本章 16 张，从硬件框图一路讲到 OpenSSL 命令行。**如果你只能听一章，听这一章。**

Slide 24 · TI HSM 硬件架构 (90 秒)

打开 HSM 子系统盒子看里面有什么：

- **Cortex-M4F @ 200MHz**——独立 CPU，不共享 R5F 的资源
- **HSM ROM + HSM SRAM (64KB 私有)**——信任锚和 ROM Code 在这里
- **DTHE**——加速器四件套：AES-128/192/256、SHA-256/512、PKA(RSA/ECC)、TRNG
- **eFuse OTP**——BMPK Hash、MEK、TIFEK、LC State、SWRV 都烧在这
- **Mailbox / IPC**——和外面 R5F MSS 域的唯一通道

关键约束：**HSM 域和 MSS R5F 域物理隔离，R5 无法直接访问 HSM SRAM/ROM/eFuse。**这不是软件限制，是硬件隔离。你在 R5 代码里写 `*((volatile uint32_t*)0xE0000000) = x` 也读不到 HSM 内部——这是整个信任模型的物理基础。

Slide 25 · HSM vs SHE / EVITA 对比 (90 秒)

四个标准摆一起：**SHE / EVITA-Light / Medium / Full。**AWR2944 对标最右侧 EVITA-Full。

念几个关键差异：

- SHE 只有 AES-128，连 SHA 都不要求；EVITA-Full 全要
- SHE 用 PRNG，EVITA-Full 用 NIST SP800-90B **真随机数**
- SHE 没有独立 CPU，EVITA-Full 有专门的 Cortex-M4F

结论：AWR2944 的 HSM 既有独立 CPU/RAM/ROM，又有完整对称+非对称+随机数，**完全满足角雷达 SecOC + Secure Boot + Secure OTA 的全套需求。**

Slide 26 · TI 设备生命周期 (100 秒)

三态单调演进：**GP → HS-FS → HS-SE。**

- **GP (General Purpose)**——无安全，任何镜像可加载，评估板/样片在这态
- **HS-FS (Field Securable)**——已烧 OEM 公钥 Hash，启用 Secure Boot，OEM 量产前调试用
- **HS-SE (Securely Enforced)**——全部密钥锁死、调试口关闭，**真正量产态**

关键警告：这条路径**单向不可逆。**eFuse 是 One-Time-Programmable，烧错就报废。**量产前必须充分验证**——我见过有同事在 HS-FS 阶段就急着锁到 HS-SE，结果发现一个小 bug，整批板子报废。

Slide 27 · TI HSM 密钥体系总览 (100 秒)

8 类核心密钥，按所有者分三组：

- **TI 出厂烧入**：TIFEK (信封加密 Keywriter 包)、TI Cert (芯片唯一身份)
- **OEM 主密钥**：BMPK/BMEK (Boot 验签/解密主密钥)、SMPK/SMEK (App 和 OTA 层)、MPK/MEK (通用用户自定义)
- **OEM 备份**：BMPK#1 / BMEK#1，撤销后启用

所有 OEM 密钥在通过 Keywriter 烧入前，**都用 TIFEK 信封加密**——这意味着即使 OTP 烧录包在传输途中被截获，攻击者也读不出明文密钥。**这套机制对客户完全透明**，第 36 张会详讲流程。

Slide 28 · CPFROM 与 eFuse 物理布局 (80 秒)

两块区域：**CPFROM 是 TI 在 wafer test 时烧入的 OTP 区**，客户动不了；**eFuse 是 OEM 自定义区**，由 Keywriter 工具一次性烧入。

CPFROM 里存：Device Type、Lot/Wafer ID、TIFEK (AES-256)、TI Public Key (RSA-3072)。

eFuse OEM 区里存：BMPK Hash (256-bit, 注意是 Hash 不是公钥本身)、BMEK、Key Revision 标志位、SWRV 计数器、Debug Lock。

节省 OTP 空间的设计：eFuse 只存 BMPK 的 Hash，完整公钥放在 X.509 证书里。这省了 RSA-3072 公钥 384 字节的宝贵 OTP 空间。

Slide 29 · 根密钥撤销机制 (85 秒)

如果某天发现 BMPRK 私钥泄漏了怎么办？**Key Revision 单调计数器 + 备份密钥。**

eFuse 里有 1 bit 的 State 标志位：**State 0 (出厂默认) 用主 BMPK + BMEK；烧 1 bit 后切换到 State 1，使用 BMPK#1 + BMEK#1。**

6 步撤销流程：准备新密钥包 → OTA 推过渡版本 → 确认 100% 安装 → 烧 eFuse 一 bit → ROM 下次启动自动切换 → 停用主密钥。

不可逆，只能撤销一次——这就是为什么要备份两把 BMPK 的根本原因。RSA-2048 安全等级过时也是触发场景之一，所以**新项目起步直接上 RSA-3072/4096。**

Slide 30 · X.509 证书是怎么来的 (75 秒)

OpenSSL 三步命令即可生成：

1. **生成密钥对**：`openssl genrsa -out priv.pem 3072`
2. **构造证书模板**：在 config 里写 OID，比如 `1.3.6.1.4.1.294.1.1 = boot_seq`

3. **签发自签名证书**：`openssl req -new -x509 -key priv.pem -out cert.pem ...`

TI 提供的 `imgtool` 或 `x509CertificateGen.sh` 把这三步打包成一条命令。**重点不是命令本身，重点是 OID 字段——下一页展开。**

Slide 31 · TI X.509 OID 扩展总表 (100 秒)

这是今天**最重要的一张**。TI 在 X.509 的 Extensions 字段下定义了私有 OID 命名空间：**1.3.6.1.4.1.294.1.x**，这串前缀意思是 iso / dod / internet / private / enterprise / TI / security / field。

七个后缀：

- **.1 boot_seq**——镜像加载序号
- **.2 image_integrity**——哈希算法+值+镜像长度+加载地址
- **.3 image_load_addr**——SoC RAM 加载地址
- **.4 swrv**——Software Revision，反回滚计数
- **.5 encryption**——加密算法+IV+密文长度
- **.6 key_derivation**——BMEK 直接用 / 派生子密钥
- **.7 debug**——Debug 控制位，JTAG 临时启用/禁用

为什么放在 X.509 里？证书是签名整体的，OID 字段被一并 hash+签名，这些 boot 元数据**天然受签名保护**，任何篡改都会导致验签失败。这是极其优雅的设计——复用了成熟的标准，不需要发明新的格式。

Slide 32 · boot_seq + image_integrity 详解 (90 秒)

OID .1 boot_seq——决定 ROM 加载哪个核。INTEGER 类型，从 0 开始升序。HSM-RT 必须 seq=0 最先加载、然后 seq=1 的 R5F-0 SBL、seq=2 的 R5F-1 应用、seq=3 的 DSP。

三个坑：① 不能重复 ② 不存在的核分配 seq 会卡死整链 ③ HSM-RT 必须 seq=0。

OID .2 image_integrity——镜像完整性元数据，SEQUENCE 结构包含 integrity_type (1=SHA512)、integrity_value (hash 值)、image_size、load_address、is_encrypted。

ROM 验签三步：① 用 BMPK 公钥验 X.509 整证书签名，拿到可信的 integrity_value ② DTHE 重新计算镜像 SHA-512，与 integrity_value 比较 ③ 一致后拷到 load_address 跳转。**加密镜像时 integrity_value 校验的是明文 hash**——先解密再 hash。

Slide 33 · swrv / encryption / key_derivation / debug 详解 (100 秒)

剩下四个 OID：

.3 swrv——5-bit INTEGER (0~31)。单调递增，ROM 验签后比对 eFuse 中已记录的最高版本号，**new < stored 则拒绝加载**。陷阱：一旦烧到 31 就无法降级，量产前严格管控版本号。

.4 encryption——SEQUENCE 含 IV 和 encryption_algo。**实际密钥 SMEK 不出现在证书里**，由 ROM 用 KEK 现场派生（见 .5）。

.5 key_derivation——TI 用 NIST SP 800-108 CMAC-KDF 派生 SMEK，用 salt + label + context 作为输入。客户每次出版本必须用新 salt → 即使 KEK 不变，SMEK 也是一次性。

.6 debug——授权调试矩阵，bit0 JTAG 全开、bit1 仅授权特定 UID 开 JTAG、bit2 允许 secure ROM trace，还有 debug_uid_check 是 SoC Unique ID 的 Hash。**量产镜像必须 debug_perm_flags=0x00 全锁。**

Slide 34 · 镜像创建 7 步流程 (90 秒)

客户构建机上从代码到安全镜像的完整流水线：

1. **编译** → GCC/TI CGT → R5F .out 二进制
2. **格式转换** → out2rprc + multicore → Multi-Core Image
3. **随机生成 SMEK** → OpenSSL rand → 32-byte 一次性密钥
4. **AES-CBC 加密** → 用 SMEK + IV 加密镜像
5. **SHA-512 哈希** → 对明文镜像算 hash → integrity_value
6. **构造 X.509 OID** → 把 boot_seq/integrity/swrv/IV/KDF/debug 填入 OID
7. **BMPRK 签名** → 用客户私钥签整证书 → 输出 .signed.bin

输出：`app_signed.bin = [X.509 证书(含 OID 元数据+BMPRK 签名)] + [AES-CBC 加密后的应用镜像]`。

Slide 35 · 镜像验证 7 步流程 (90 秒)

上电后 HSM ROM 在芯片内的 7 步验证，**任何一步失败立即停止**：

1. **解析头部** → 格式校验
2. **公钥认证** → 用 eFuse 中 BMPK Hash 比对证书内嵌公钥的 SHA-512
3. **证书验签** → 用 BMPK 公钥 RSA-PSS / ECDSA 验整证书签名
4. **OID 提取** → 解析 boot_seq / integrity / swrv 等
5. **反回滚检查** → swrv ≥ eFuse 中版本号
6. **解密镜像** → DTHE 用派生的 SMEK AES-CBC 解密
7. **完整性校验** → 重算 SHA-512 与 integrity_value 比对

失败处理极其保守：ROM 立即停止，**不记录失败原因到 RAM、不点亮任何外设引脚、直接看门狗复位**——攻击者无法通过失败信息推测密钥。

最后一道 **Magic Number 0xAABBCCDD** 防止跳转到伪造或残留代码段。**典型耗时全链 < 30ms**（DTHE 硬件加速）。

Slide 36 · OTP Keywriter + TIFEK 信封加密 (85 秒)

量产线场景：**客户的密钥必须烧入 eFuse，但工厂电脑、产线员工都不可信**——怎么办?

三阶段流程：

- **客户侧**：生成密钥，用 TIFEK_Pub 加密整个密钥包 → 输出 .enc 文件 (公开传输安全)
- **TI 工具**：客户把 .enc 提交给 TI 在线工具，生成 Keywriter Image (内嵌加密密钥包 + 烧录指令 + TI 签名) → 输出 keywriter.signed.bin
- **工厂线**：通过 JTAG 加载，HSM ROM 验签后进入执行态，用 TIFEK_Priv (eFuse 中) 解密还原明文密钥，**密钥仅在 HSM 内部传输到 eFuse**，整线员工/PC 看到的全是密文

TIFEK = TI Factory Encryption Key，TI 出厂时已烧入每颗芯片 HSM，客户无法读取也无需管理。**整套机制对客户透明**——这是极其聪明的"零信任"密钥分发设计。

Slide 37 · 安全调试 WIR / SECAP / RMA (75 秒)

量产后 JTAG 还能用吗？设备坏了怎么 RMA 回 TI？三层机制：

- **WIR Mode**：跟随设备 LC 变化，GP 全开、HS-FS 可控、HS-SE 默认锁。HS-SE 状态下 JTAG **只能通过 X.509 中 .6 debug 字段授权才能临时打开**
- **SECAP (SECure Debug Authentication Protocol)**：5 步挑战应答，Debug 主机请求 JTAG → HSM 发送 Challenge → 主机用授权私钥签 Challenge+UID → HSM 验签解锁。**授权私钥仅签该 SoC 的 UID，钥匙不可移植到其他芯片**
- **RMA Flow**：5 步销毁，TI 给客户 RMA 工具，用 TI 私钥签 Erase 命令，HSM ROM 验证后擦 eFuse 中客户密钥，锁回 GP-like 状态寄回 TI

三层防线串起来：WIR 限定调试上下文 → SECAP 在线挑战应答 → RMA 安全报废。

Slide 38 · 软件层面如何实现 (90 秒)

三件事做好：

① OpenSSL 工具链——genrsa 生成 BMPRK、dgst 算公钥 Hash 烧 eFuse、req -new -x509 签镜像。所有 OID 写在 x509_template.cnf 里，每次出版本只改 swrv 字段。

② CI/CD 自动签名流水线——build → sign → test → archive 四阶段。关键安全约束：**BMPRK 私钥仅在签名服务器 HSM 中、不出网、CI 通过 PKCS#11 调签名服务而非传 .pem 文件。**

③ 反回滚 SWRV 烧 eFuse——每次验签通过后把当前 swrv 写回 eFuse 记录的最高版本号。陷阱：**swrv 5-bit 上限 = 31，一次烧错整批芯片永久无法降级。**量产首版常用 swrv=2 或 3，留低位空间给紧急回滚。

第四章小结 · 过渡到第五章 (15 秒)

TI 这一套讲完了。最后 8 分钟快速过一下——**这些通用机制到我们公司的工程里，具体是怎么对接的。**

第五章 · 公司平台落地 (8 分钟)

Slide 39 · 第五章封面 (10 秒)

这 9 张的目的不是重讲 TI，而是**把通用机制对齐到博世工程实现。**

Slide 40 · 两层 HSM 概念厘清 (60 秒)

很多同事经常把两层搞混，今天在这里彻底澄清：

- **Layer 1 TI HSM ROM**——**芯片硬件层**，固化在 Mask ROM 里，不可改。负责 Secure Boot / 密钥管理 / X.509 验签 / DTHE 加解密。状态机是 GP → HS-FS → HS-SE
- **Layer 2 博世 HSM-RT**——**应用层固件**，跑在 Cortex-M4F 上，客户可定制。负责应用层加密服务 / 安全存储 / SecOC / 自定义 LC。状态机是 Init → Field → Compromised → Decom

关键关系：HSM-RT 是 TI HSM 硬件上跑的应用程序，由 boot_seq=0 加载。**一个是硬件的，一个是软件的；一个是 TI 控制的，一个是博世控制的。**

Slide 41 · HSM-RT 应用层 LC 状态机 (50 秒)

四个态：**Init → Field → Compromised → Decom**。

- Init 首次上电，无业务密钥
- Field 正常运行态，99% 时间在这
- Compromised 检测到入侵，业务密钥不可用仅允许诊断/OTA 救援
- Decom 永久退役，密钥全清

转移条件：OEM 密钥注入完成进 Field；MAC 持续失败 / 时钟异常 / 调试探测触发 Compromised；OTA 推清白镜像可恢复 Field；上层指令 + 安全令牌不可逆进 Decom。

最重要：**这套状态与 TI eFuse 硬件状态完全独立**——一个是 RAM/Flash 标志位，一个是 OTP 不可逆。**别再混了。**

Slide 42 · TI 真实启动序列 (60 秒)

5 段链：**HSM ROM → R5 ROM → R5 SBL → HSM-RT → App**。

- HSM ROM 硬件复位起点，TI 固化
- R5 ROM 唤醒主核，由 HSM ROM 加载并验签
- **R5 SBL 是 TI 的 Secondary Bootloader 模板——这就是我们公司 BM 所在的槽位**
- HSM-RT 在 HSM Cortex-M4F 上跑
- App 是 R5F/DSP 上的雷达感知代码

整链 5 段，4 次验签，全链可信。**原版 Slide 27 没有映射到 TI 真实链**，这张是彻底重画的。

Slide 43 · 公司二级 Bootloader 映射 (55 秒)

TI 视角的 R5 SBL 槽位，在博世内部是一个四层套娃：

BM (TI 模板最外层) ⊃ PFBoot (Platform Boot 选 App slot) ⊃ CustBoot (解析升级标志位) ⊃ CustApp (业务代码)

每一层都被前一层 BMPK 验签。整链 5 段共 4 次验签，启动时间多 < 80ms (DTHE 加速)。

对外汇报时：把 BM/PFBoot/CustBoot 统称为"公司 Bootloader"；**对内设计时**：这三层职责划分清晰，PFBoot 管硬件，CustBoot 管软件升级。

Slide 44 · Secure Flash 布局 (60 秒)

按 Block 号划分：

- **Block 0**：X.509 证书 + Hash 表，复位起点
- **Block 1-8 固定**：BM、PFBoot、CustBoot、HSM-RT 各占 2 块，都被 BMPK 签
- **Block 9-40 双 App 互斥 ★ 易混点**
- **Block 41-50**：雷达标定数据，MAC 保护
- **Block 51-63**：日志 & NVM，HMAC-SHA256

重点说 Block 9-40 的互斥共存：同一段区域在不同生命周期承载不同应用。EOL 测试 / 量产线烧 ProductionApp 跑自动化测试；量产交付前由 ProductionApp 自擦 + 写入 CustApp → CustApp 上电启动。**二者绝不可同时存在**——X.509 证书 OID 中只有一份 boot_seq，残留会导致 ROM 加载冲突。这条是**原版漏讲的关键约束**。

Slide 45 · Secure Flash 操作流程 (50 秒)

读写都走 HSM-RT API：

- **写**：App 调 `secure_flash_write` → HSM-RT 校验调用方权限 → DTHE 计算 HMAC-SHA256 → 写入 `[data || HMAC]` → 更新元数据
- **读**：App 调 `secure_flash_read` → HSM-RT 读整块 → 重算 HMAC 比对 → 匹配则返回
- **异常**：HMAC 失败标记 Block Corrupted → HSM-RT 进 Compromised 态 → 上报 OEM

掉电中断对策：**双 Bank 写入 + 元数据原子更新。**擦除限制：Block 0-8 永不可擦；Block 9-40 替换需配 X.509 重签。

Slide 46 · Secure OTA · 差分升级 + 反回滚 (55 秒)

云端到 ECU 的 7 段时序：**Server → TLS 通道 → Gateway → ECU CAN-FD → PFBoot Patch Apply → X.509 验签 → 切槽切版本。**

A/B 双槽切换：Slot A 跑 v1.5、Slot B 存 v1.4 备份。OTA 写入 Standby 槽 → 验签通过 → 翻 A/B 标志位 → 下次启动 ROM 选新槽。**启动失败可立即翻回旧版。**

反回滚强制：`if (new_swrv >= cur_swrv) apply_patch; else reject_OTA`。关键约束：**A/B 槽各自独立 swrv，回滚不破反回滚；OTA 服务器永远只允许 swrv 递增；紧急召回出新版本把回滚版打包进去。**

Slide 47 · 总结与展望 · Q & A (60 秒, 含问答引导)

今天我讲清了三件事——汽车信息安全的攻击面和标准体系、TI AWR2944 HSM 的硬件能力和 X.509 OID 全机制、博世 HSM-RT 应用层到 Secure Flash / OTA 的落地。

做项目时记住三条金律：

1. **TI 硬件 LC 和博世应用 LC 是两层，别混**
2. **swrv 5-bit 上限 31，量产前规划版本节奏**
3. **CustApp 与 ProductionApp 在 Block 9-40 互斥共存**

技术演进趋势也聊一下：**RSA-3072 + ECC-256 → 后量子密码 PQC**；**ISO/SAE 21434 + UN R155/R156 全球落地中**；**HSM 从 SHE 走向 EVITA-Full + AI 异常检测。**

好，讲到这里。**欢迎围绕 X.509 OID 设计、HSM-RT API 调用、Secure OTA 工程化、反回滚版本管理这四个方向提问。**谢谢大家。

讲者节奏参考

章节	幻灯片	时长	节奏建议
开场	1	0:30	快速热身, 明确目标
目录	2-4	0:45	标注修订点即可
Ch1 基础	5-10	10:00	Slide 8 经典案例可互动
Ch2 核心技术	11-16	12:00	Slide 12/14 是后续基础, 务必讲透
Ch3 雷达	17-22	10:00	Slide 18 STRIDE 可以点名举例

章节	幻灯片	时长	节奏建议
Ch4 HSM 实战	23-38	20:00	**Slide 31-35 是全场重点, 多给点时间**
Ch5 落地	39-47	8:00	节奏加快, 作为总结
Q&A	47	预留	4 个方向引导

常见提问应对:

- **问: RSA 会不会被量子计算机破解?** 答: 10-15 年内可能, 所以新项目直接 RSA-3072 起步, 留迁移窗口给 PQC (ML-KEM / ML-DSA)
- **问: 如果 BMPRK 真的泄漏了?** 答: 走 Key Revocation 流程, 切到 BMPK#1, 这就是为什么必须烧两把
- **问: 调试用的 JTAG 是不是彻底没用了?** 答: 量产机器可以, 通过 SECAP 协议 + 特定 UID 授权, 但只能签给特定芯片不可移植
- **问: swrv 烧满 31 怎么办?** 答: 换芯片重烧, 没有软救措施——所以量产前版本规划极其重要
- **问: HSM-RT 和 AUTOSAR CSM 是一回事吗?** 答: CSM 是 AUTOSAR 抽象接口, HSM-RT 是具体实现, 我们的 CSM 底层就是调 HSM-RT API